ARSON Hugo CHAPUIS Guilhem DE TOMA Matteo

M1, 2ème année du Magistère de Physique Fondamentale d'Orsay

UE Informatique Avancée - orientée objet taught by KEBAÏLI Nouari

AF447: An avoidable catastrophe ?

Recoverability study using longitudinal modeling

Abstract

On the evening of May 31, 2009, at 22:29, an Air France Airbus A330 took off from Rio de Janeiro/Galeão International Airport bound for Paris Charles de Gaulle. A few hours later, the aircraft crashed into the Atlantic Ocean. Data from the flight recorders indicate that the aircraft entered a high-altitude upset, stalled, and was not recovered.
In this work, a longitudinal flight-dynamics model was developed to investigate whether recovery remained physically achievable under the conditions of the event. The pilot-command and automatic rescue strategies were tested and compared in order to evaluate their ability to restore a recoverable trajectory. A genetic algorithm was finally developed to create an automatic rescue scenario that could easily be generalised to different flight and stall conditions.

Résumé

Le 31 mai 2009 au soir, à 22 h 29, un Airbus A330 de la compagnie Air France décolle de l'aéroport international de Rio de Janeiro/Galeão à destination de Paris Charles-de-Gaulle. Quelques heures plus tard, l'appareil s'est abîmé dans l'océan Atlantique. Les données des enregistreurs de vol indiquent que l'avion est entré dans une perte de contrôle à haute altitude, a décroché, puis n'a pas pu être récupéré.
Dans ce travail, un modèle de dynamique du vol longitudinal a été développé afin d'étudier si une récupération restait physiquement possible dans les conditions de l'événement. Les commandes du pilote et des stratégies de sauvetage automatique ont été testées et comparées afin d'évaluer leur capacité à ramener l'appareil vers une trajectoire récupérable. Un algorithme génétique a finalement été mis au point afin de créer un scénario de récupération automatique pouvant être facilement adapté à différents avions et conditions de décrochage.

Introduction

The goal of this project is to model aircraft stall dynamics and to investigate whether an effective recovery strategy can be identified. Our study is motivated by the 2009 crash of Air France Flight 447 (Paris–Rio), which remains a major reference case for the analysis of high-angle-of-attack flight and loss-of-control scenarios.
To address this question, we developed a reduced longitudinal flight-dynamics model restricted to the vertical plane (x, z). Our approach focuses on the essential coupled dynamics between forward and vertical motion, and pitch evolution, which are the dominant mechanisms involved in stall entry and recovery in the scenario considered here. Individual contributions are detailed in Appendix I.

Fig. 1: The Airbus A330 which crashed into the ocean, photographed in March 2007

1 Theoretical Basis

1.1 Reference Frames and States

In this work, all the variables used are given in standard SI units. In an effort of lisibility, the units are thus always omitted.
We use a simple right-handed inertial frame $\mathcal{I}$ Earth-fixed with cartesian axes (x_I, z_I) in the vertical plane; z_I is positive upward. The state container in the code is the structure `EtatCinematique`, which defines the following variables and units:

$$\mathbf{x} = \begin{bmatrix} x & z & v_x & v_z & \theta & \omega_\theta \end{bmatrix}^T$$

where x, z are position components, v_x, v_z are velocity components, θ is the pitch angle, and ω_θ is the pitch rate.
Only the pitching moment is modeled; roll and yaw dynamics are neglected. In the BEA report, we observe that those two angles exhibit small oscillations around 0 (except in occasional moments), hence their impact on the lift force and stall dynamics can be neglected at first approximation.

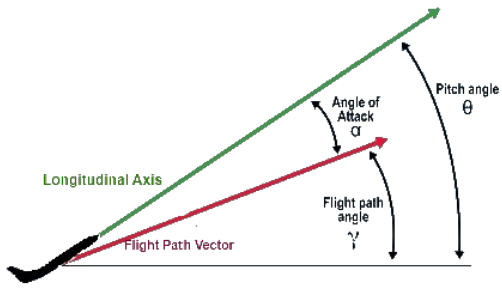


Fig. 2: Definition of flight angles relative to the horizontal

1.2 Forces and Moments Balance

The dynamics in the longitudinal plane are governed by Newton's second law and Euler's rotational equation (all written in SI units). We project forces (thrust T, drag D, lift L, weight W) onto the inertial axes (x, z) using pitch angle θ and flight-path angle γ, as shown in Fig. 2. Thrust acts along the fuselage, drag opposes velocity and lift acts perpendicular to the velocity. Then:

$$F_x = T\cos\theta - D\cos\gamma - L\sin\gamma$$
$$F_z = T\sin\theta - D\sin\gamma + L\cos\gamma - W$$

The translational equations are then:

$$\dot{v}_x = \frac{F_x}{m}$$
$$\dot{v}_z = \frac{F_z}{m}$$

where m is the aircraft mass. Over the $\approx 5\,\text{min}$ time window considered in this study, fuel burn is limited (about 800 kg, i.e. less than 0.5 % of the aircraft mass), so the mass variation has a negligible impact on the longitudinal dynamics and m is assumed constant.

The pitching moment M_θ consists of aerodynamic (M_{aero}) and thrust-induced moments (M_{thrust}).
The thrust moment arises from the vertical offset z_t between the engine and the center of mass:

$$M_{\text{thrust}} = z_t \cdot T$$

Euler's rotational equation for pitch is:

$$I_\theta \, \dot{\omega}_\theta = M_\theta$$

where I_θ is the aircraft moment of inertia about the pitch axis.

1.3 Aerodynamic Coefficients

Aerodynamic forces and moments are computed from the coefficients of lift, drag, and pitch moment:

$$\begin{aligned} L &= \tfrac{1}{2}\rho V^2 S \, C_L(\alpha, \delta_p), \\ D &= \tfrac{1}{2}\rho V^2 S \, C_D(\alpha, \delta_p), \\ M_{\text{aero}} &= \tfrac{1}{2}\rho V^2 S \, l \, C_m(\alpha, \delta_p, \omega_\theta), \end{aligned}$$

where:

- ρ is the air density
- V is airspeed
- S is the wing reference area
- l is the mean aerodynamic chord
- δ_p is the elevator control surface deflection

1.3.1 Lift coefficient

In the attached-flow regime ($\alpha \leq 13°$), the lift coefficient follows a linear relationship:

$$C_L(\alpha \leq 13°, \delta_p) = C_{L_\alpha} \, (\alpha - \alpha_0) + C_{L_{\delta_p}} \delta_p$$

where C_{L_α} is the lift-curve slope, α_0 is the zero-lift angle of attack, and $C_{L_{\delta_p}}$ is the elevator effectiveness [Mod].
For $\alpha > 13°$, the model transitions into the stall regime. Flow separation over the wing reduces lift generation. We model this post-stall region using a parabolic decay function:

$$C_L(\alpha > 13°) = A(\alpha - \alpha_{\text{peak}})^2 + C_{L,\max}$$

where $\alpha_{\text{peak}} = 15°$ is the angle of maximum lift, $C_{L,\max}$ is the peak lift coefficient, and A is chosen to ensure tangency with the linear curve at $\alpha = 13°$. This parabolic model and the choice of $\alpha_{\text{stall}} \approx 13°$–$15°$ is consistent with transport aircraft behavior, as documented in BEA report [Bur12a]. The lift coefficient is clamped to non-negative values for $\alpha > 22°$ where the model reaches zero.

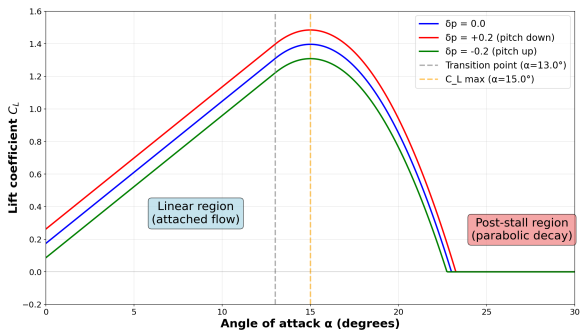


Fig. 3: Lift coefficient as a function of angle of attack and elevator control surface deflection.

1.3.2 Drag coefficient

The drag coefficient combines zero-lift (parasitic) drag and induced drag in the attached-flow regime:

$$C_D(\alpha \leq 13°) = C_{D_0} + K C_L^2$$

where C_{D_0} and K are given in [Mod].
In the post-stall regime ($\alpha > 13°$), separation-induced form drag dominates. We increase the baseline parabolic drag with a $\sin^2$-based correction:

$$C_D(\alpha > 13°) = C_{D,\text{ref}} + 2\left[\sin^2(\alpha) - \sin^2(\alpha_{\text{ref}})\right]$$

where $C_{D,\text{ref}}$ is the drag at the transition angle $\alpha_{\text{ref}} = 13°$. The $\sin^2$ term models the increased pressure drag from the bluff-body behavior of the stalled wing (see [Wic54]). A similar correction applies for negative stall angles ($\alpha < -4°$).

1.3.3 Pitching Moment Coefficient

The aerodynamic pitching moment about the center of gravity is modeled as the sum of three effects: the static restoring moment, the elevator contribution, and the pitch-rate damping term. According to [Mod], this leads to

$$C_m(\alpha, \delta_p, \omega_\theta) = C_{m_0} - C_{m_\alpha}(\alpha - \alpha_0) - C_{m_{\delta_p}}\delta_p - C_{m_q}\frac{\omega_\theta l}{V}$$

In this expression, C_{m_0} denotes the baseline pitching-moment coefficient at reference incidence, while the term proportional to $(\alpha - \alpha_0)$ represents longitudinal static stability: when the angle of attack departs from its reference value, the aircraft develops a restoring pitching moment whose intensity is governed by C_{m_α}.
The third term accounts for elevator action. A nonzero deflection δ_p modifies the pitching moment with a control authority measured by $C_{m_{\delta_p}}$.
Finally, the last contribution introduces pitch damping through the angular rate ω_θ. This term opposes rapid pitching motion and therefore plays a stabilizing role in the transient response.
The numerical values of all these aerodynamic derivatives are taken in [Mod].

1.3.4 Implementation and Validation

The aerodynamic model is implemented in the `ModeleLineaire` class. Coefficients are updated at each integration time step based on the instantaneous angle of attack $\alpha = \theta - \gamma$. Elevator deflection δ_p is computed from pilot commands through the `SystemeControle` interface, with saturation limits to represent physical actuator constraints.

1.4 Propulsion and Control Inputs

The available thrust is calculated as a function of altitude via the relative atmospheric density $\sigma = \rho/\rho_0$ where the atmospheric density (at 1 atm, 15°C) $\rho_0 = 1.225\ \mathrm{kg \cdot m^{-3}}$:

$$F_{\max}(z) = n_{\text{engines}} \cdot F_0 \cdot \sigma$$

with $F_0 = 300{,}000$ N the nominal thrust per engine at sea level and $n_{\text{engines}} = 2$. The effective thrust is then commanded by a normalized signal:

$$F_{\text{thrust}} = c_T \cdot F_{\max}(z), \quad c_T \in [0, 1]$$

The elevator deflection δ_p is obtained by superposition of a trim angle and a commanded increment:

$$\delta_p = \delta_{p,\text{trim}} + c_p \cdot \delta_{p,\max}$$

where $c_p \in [-1, 1]$ is the normalized pitch command (-1 = maximum nose-up, $+1$ = maximum nose-down), $\delta_{p,\max}$ represents the maximum control authority, and $\delta_{p,\text{trim}}$ is the equilibrium deflection defined in Section 2.2. Indeed, in real aircraft operation, the pilot (or autopilot) does not directly command an absolute elevator angle directly, but rather a control input relative to a trimmed equilibrium.

1.5 Atmospheric conditions

To compute atmospheric quantities in the simulator (temperature T, pressure p, and density ρ), we adopt the ISA framework shown in Fig. 4. The model describes how the atmosphere varies with altitude in two successive regimes [Cav00].

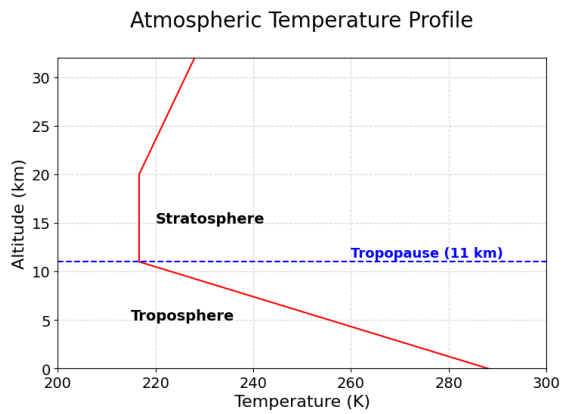


Fig. 4: International Standard Atmosphere (ISA)

At low altitude, up to the tropopause $h_t \approx 11\,000$ m, we are in troposphere. In this layer, temperature is assumed to decrease linearly with altitude, with lapse rate L:

$$T(z) = T_0 - Lz, \qquad 0 \le z \le h_t.$$

Once temperature is known, density is obtained from the hydrostatic + perfect-gas ISA relation:

$$\rho(z) = \rho_0 \left(\frac{T(z)}{T_0}\right)^{\frac{g}{RL}-1}.$$

Above the tropopause, in the lower stratosphere, the model assumes an isothermal layer. The temperature is therefore constant at its tropopause value:

$$T(z) = T_t, \qquad z > h_t.$$

With constant T, density no longer follows a power law and instead decays exponentially:

$$\rho(z) = \rho_t \exp\left(-\frac{g}{RT_t}(z - h_t)\right).$$

In these expressions, T_0 and ρ_0 denote sea-level temperature and density, g is gravity, R is the specific gas constant for air, and L is the tropospheric temperature lapse rate (ISA value: $L \approx 6.5 \times 10^{-3}\,\mathrm{K\,m^{-1}}$). T_t, ρ_t are the temperature and density evaluated at $z = h_t$ to ensure continuity between the two layers.
The ISA model works up to 100 km, but considering the flight envelope, we do not consider atmospheric variations above 20 km.

2 Numerical Integration and Simulation

2.1 Numerical Integration

The simulation employs a fourth-order Runge-Kutta (RK4) integration scheme by default, with the option to use explicit Euler integration for comparison. The choice of method is controlled via the method `Param_Simulation::methode_integration` in the `Constantes.h` file and can be chosen by the user in `Config.txt`.
A fixed time step of $\Delta t = 0.01$ s is used for all simulations. The reliability of the integration scheme and the time step were verified using conservation of energy (Section 2.3). Both methods yield equivalent results in standard conditions, with differences appearing only in pathological edge cases.

2.2 Initial Conditions and trim

Simulations are initialized at steady cruise conditions. The initial state must satisfy force and moment equilibrium.
The trim state is computed iteratively via the `CalculateurTrim` class, which solves the coupled nonlinear system:

$$\begin{aligned} L(\alpha, \delta_p, V) - W &= 0 \\ M_{\text{aero}}(\alpha, \delta_p, V) + M_{\text{thrust}} &= 0 \\ T(c_T) - D(\alpha, V) &= 0 \end{aligned}$$

Since the cruise trim depends on the flight conditions, we solve, at a prescribed cruise altitude and target speed V, this system with unknown trim variables α_{trim}, $\delta_{p,\text{trim}}$, $c_{T,\text{trim}}$ required to maintain equilibrium.
The process iterates until convergence criteria $|L - W| < 10^{-6}$ N and $|M_{\text{total}}| < 1000$ N $\cdot$ m are satisfied.
Steady cruise conditions are defined as:
$z = 10\,670.0$ m and $v_x = 240.0$ m/s. The calculated trim is $\delta_{p,\text{trim}} \approx -0.11$ rad and the incidence is $\alpha \approx 4.4°$, which is approximately what we observe on the black boxes data [Bur12b].

2.3 Simulation Pipeline

The simulation workflow is configuration-driven: all physical parameters, numerical settings, control commands, and activated operations are read from `Config.txt` through the global `Config` object (`config.chargerDepuisFichier(...)` and `config.completer()` in `main.cpp`).

For a standard flight run (`SIMULATION` operation), the sequence is as follows.

1) Scenario setup from configuration
The run is initialized from `Config.txt`: geometry (`surface`, `corde`), mass (`masse`), initial state (`vx_ini`, `z_ini`), numerical parameters (`dt`, `duree`), command schedule, rescue options, and output paths are loaded in `main.cpp`.
An `Avion` instance is then created as the central physical model. This class is built as a composite object that encapsulates the main subsystems required by the simulation:

`ModeleAerodynamique` (aerodynamic coefficients and stall behavior), `Propulsion` (thrust model), `SystemeControle` (pilot/autopilot normalized commands), `Environnement` (ISA atmosphere quantities), `ProprietesInertie` (mass/inertia data), and `EtatCinematique` (state variables).
Calling `avion.initialiser(vx_ini, z_ini)` sets the initial kinematic state consistently with these coupled subsystems before trim and time integration.

2) Trim computation and initialization
Before time marching, a trim is computed with `CalculateurTrim`. These trim values are applied to initialize the aircraft state and control references consistently.

3) Time integration loop
The `Simulateur` advances the dynamics over $t \in [0, t_f]$ with fixed step $\Delta t =$ `dt` (typically 0.01 s). At each step, it: updates control inputs (including perturbation windows and optional rescue logic), computes forces/moments through the aerodynamic and propulsion models, and integrates the state with the selected method (Euler or RK4 by default).

4) Optional rescue logic
If `enable_rescue_system = true`, `SauvetageAvion` monitors critical conditions (altitude, descent rate, pitch) and applies the selected strategy (`THRUST_FIRST`, `PROFILE_FIRST`, `SIMULTANEOUS`, or `GEN_GIVE`). The rescue activation threshold and strategy are read from configuration. The rescue strategies are described in detail in Section 3.3.

5) Termination and logging
The loop stops at final time or crash condition (ground contact). During the run, `Simulateur` writes a CSV time history (kinematics, forces, moments, aerodynamic coefficients, angle of attack, and control commands).

To check the reliability and coherence of the model, we define a class `AnalyseurEnergie` to calculate the energy of the plane and the losses and compare it to the work of the different forces. We indeed observe that the two quantities are equal. We also observe that the RK4 and Euler methods are generally equivalent except for a few pathological cases.

3 Main results

3.1 Normal flight conditions validity

The first validation step is to verify that the aircraft exhibits the expected dynamics during unforced flight. In the absence of control inputs or perturbations, the stable trajectory of an aircraft exhibits an oscillation with amplitude on the order of a few tens of meters and a period on the order of one minute, called a phugoid, and is displayed in Fig. 5.

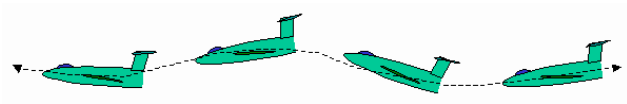


Fig. 5: Illustration of a phugoid

We initialized the aircraft in trimmed cruise condition and allowed it to evolve freely under the influence of gravity and aerodynamic forces. The results, presented in Figure 6, confirm the expected phugoid behavior.

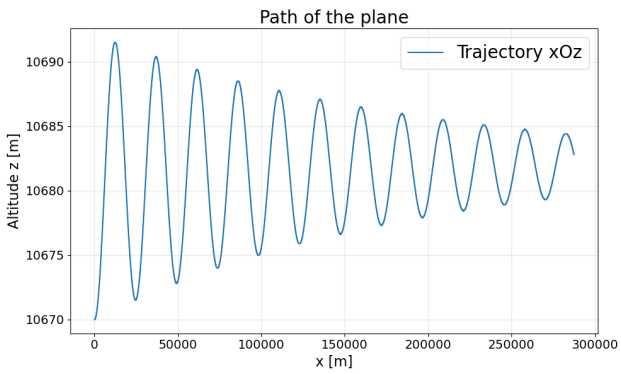


Fig. 6: Simulated phugoid oscillation

3.2 Effect of flight control inputs

In the AF447's event, after the autopilot's disconnection caused by speed probe (Pitot tube) icing, which led to unreliable airspeed indications, the pilot initiated a nose-up input (elevator deflection) by pulling on the side stick.

This is modeled by setting : $\delta_p = \delta_{p,\text{trim}} + c_p \cdot \delta_{p,\max}$. The speed of activation of the input follows an exponential law $c_p(t) = \left(\frac{t}{\tau}\right)^2 c_{\text{goal}}$, with $\tau = 45\,\text{s}$ to be consistent with black-box recordings (see Fig. 7). There is still a difference between the two curves, but the global flight dynamics is the same (climb followed by stall).

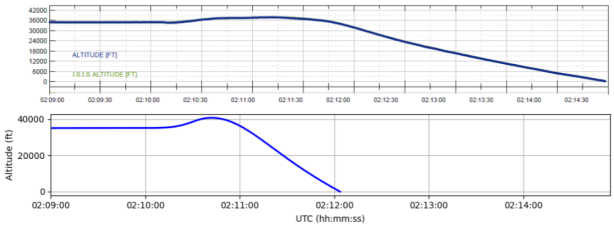


Fig. 7: Comparison of altitude over time between the black boxes (top) and the model (bottom).

Fig. 8 further illustrates the post nose-up input evolution of the trajectory and longitudinal angles. After input activation, the pitch angle θ increases rapidly, while the angle of attack α rises with a short delay and then remains at a high value (around 30°). In parallel, the flight-path angle $\gamma = \theta - \alpha$ first shows a brief climb tendency, then becomes strongly negative: the aircraft keeps a nose-up attitude while the actual trajectory points downward.

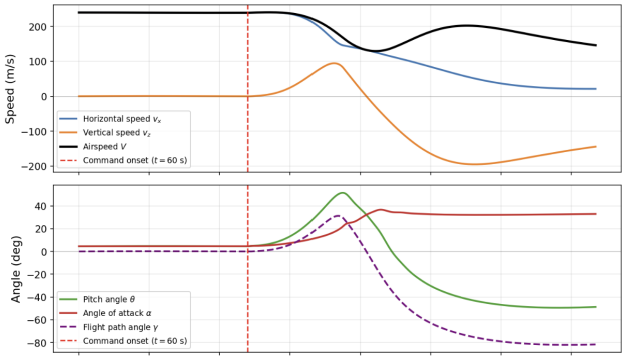


Fig. 8: Evolution over time of speeds and characteristic flight angles during the simulation

This attitude/trajectory decoupling is precisely the key feature documented in BEA report: sustained nose-up attitude, high angle of attack, and continued descent.
The minimum lift-sustaining speed is given by

$$V_{\min} = \sqrt{\frac{2W}{\rho S C_{L,\max}}},$$

where $C_{L,\max}$ is the maximum lift coefficient. Once $V < V_{\min}$, level flight is no longer aerodynamically sustainable. In our simulation, increasing the nose-up input results in an increase in both induced drag and angle of attack, accelerating kinetic-energy loss until this threshold is crossed, after which recovery becomes extremely difficult without first reducing α and rebuilding airspeed.

To better understand the inherent controllability of the aircraft, we explored the outcome of the flight as a function of both elevator input intensity (c_p) and thrust setting (c_T), holding all other parameters fixed to the accident scenario conditions. The results are shown in Fig. 9.

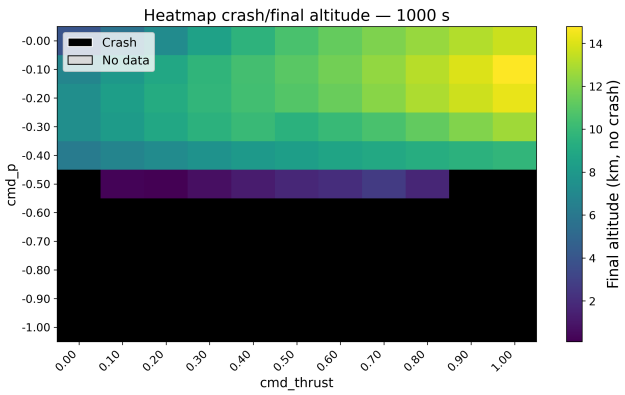


Fig. 9: Feasibility heatmap: crash, or final altitude as a function of normalized elevator and thrust commands over 1000 s

A striking observation emerges: aggressive nose-up inputs (high $|c_p|$) invariably lead to crashes regardless of thrust, because they induce excessive angle of attack, exceeding the aerodynamic stall boundary.

Conversely, moderate nose-up inputs combined with sufficient thrust allow the aircraft to maintain or gain altitude. This region represents the carefree envelope for cruise flight.

This controllability landscape indicates that the pilot's first critical error was the initial aggressive stick-pull, which rapidly drove the aircraft toward stall-prone conditions from the very beginning of the event.

3.3 Rescue attempt after stall

The next step is to assess whether the aircraft can be recovered in a stall scenario by modifying pilot reaction.
To study this, we implemented a `SauvetageAvion` class with three recovery procedures:

- **Thrust-first**: first reduce thrust, then reduce pitch, then stabilize.
- **Profile-first**: first reduce pitch, then reduce thrust, then stabilize.
- **Simultaneous**: reduce pitch and thrust together, hold, then stabilize.

For these strategies, the duration of each step in the procedure is determined by hand-tuned parameters. The results are shown in Fig. 10.

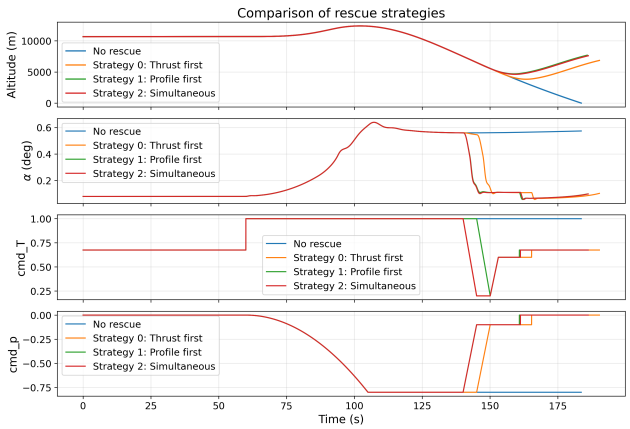


Fig. 10: Comparison of the 3 different strategies

We observe that all three strategies successfully recover the aircraft, and that in each case the angle of attack is brought back below the stall boundary. However, the Thrust-first strategy appears less efficient than the two others, as it requires more time to reduce α and restore a positive flight-path angle. This observation is consistent with the standard stall-recovery guidance described in [Air10], where the first objective is to reduce angle of attack (nose-down command).

3.4 Genetic algorithm

To design a rescue law without hand-tuning each command profile, we use a genetic optimization process that automatically converges, over generations, toward an optimized strategy (=chromosome).
Algorithmically, each generation follows the same loop, described in Appendix III :

run simulation over all chromosomes → evaluate → rank/select → recombine → mutate → next generation simulation

At each generation, all the (`Nbr_chr`) chromosomes are simulated, assigned a fitness, and ranked. The best individuals are retained as parents, recombined, and (`MutationRate_times100`)% of their genes are mutated to form the next population. After (`Nbr_generation`) iterations, the best chromosome is exported and can be reused as a fixed rescue law in `GEN_GIVE` mode for independent validation. In fact, the best chromosome is exported at each generation, overwriting the best chromosome of the last step, in order to prevent the loss of hours of computation over a power outage.
To analyze convergence, we track the best chromosome at each generation and plot four indicators on Fig. 11.

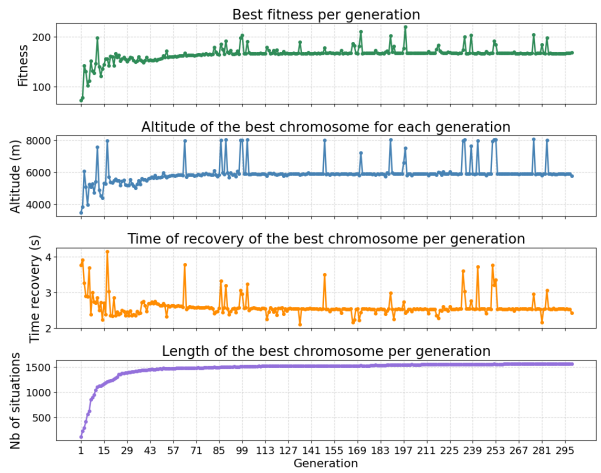


Fig. 11: Genetic algorithm convergence diagnostics (best individual per generation). From top to bottom: best fitness, recovery altitude, recovery time, and chromosome size (number of learned state–action situations).

The curves show a rapid early improvement followed by stabilization, with a slight long-term increase in fitness and altitude, nearly constant recovery time, and progressive growth of chromosome size, linked to the number of situations in which the plane have a known behavior.

Conclusion

In comparison with the BEA report, our study reaches a similar central conclusion: within the limits of the modeled dynamics, the aircraft was not irreversibly lost due to physics alone, and recovery remained theoretically possible under an appropriate control response. In that sense, our simulations support the idea that the aircraft was still recoverable.

This conclusion must however be treated with caution. The present model does not capture all aspects of the real event, in particular the possible degradation of control effectiveness at very high angle of attack. In deep stall conditions, reduced elevator authority and highly nonlinear aerodynamic effects could significantly alter recovery dynamics, potentially delaying or limiting the effectiveness of a nose-down input.

Within these limitations, our results indicate that the persistence of the stall is primarily linked to the control inputs that were applied. Stall recovery in the model requires a clear and sustained reduction of angle of attack through nose-down commands. According to official accident analyses, such inputs were not maintained; instead, prolonged nose-up commands were applied, increasing the angle of attack and preventing exit from the stalled regime. This behavior appears as a determining factor.

In other words, the issue was not only whether recovery was physically possible, but that the control response remained persistently opposite to that required for recovery, preventing any sustained reduction in angle of attack.

Future work should extend the model to include degraded control effectiveness in stall, stronger aerodynamic nonlinearities, and a fully coupled 6-DOF framework.

To conclude, through this project, we learned how to structure and coordinate a group simulation much more effectively. We also realized how important it is to define clear interfaces, and validation steps from the very beginning. In future projects, we will apply these good reflexes early.

Availability of the simulation

The different algorithms designed as part of this study are freely accessible and can be found at : *https://github.com/GCscol/AvionCrashCppM1S7*
Instructions on operating the code via the `Config.txt` interface, as well as the `'make run'` and `'make plot'` commands, can be found in the `Read_me` file attached to the algorithms.
Finally, all classes are evolutive. This code can be used as a foundation for further analysis and improvement, such as including localised turbulence.

References

[Air10] Airbus Customer Services, Toulouse, France. Flight Operations Telex: ATA 27 Stall Recovery Procedure, fot 999.0044/10 edition, May 2010. Official Airbus stall recovery procedure for all Airbus aircraft.

[Bur12a] Bureau d'Enquêtes et d'Analyses (BEA). Rapport final. Technical report, Le Bourget, France, July 2012.

[Bur12b] Bureau d'Enquêtes et d'Analyses pour la sécurité de l'aviation civile. Annexe 3 : Planches de paramètres. Technical report, BEA, 2012. Annexe du rapport final sur l'accident du vol Air France 447 (AF447), Airbus A330-203, immatriculé F-GZCP, survenu le 1er juin 2009.

[Cav00] M. Cavcar. The international standard atmosphere (ISA). Technical report, Anadolu University, 2000.

[Mod] Modèle d'un avion de transport de type airbus a300.

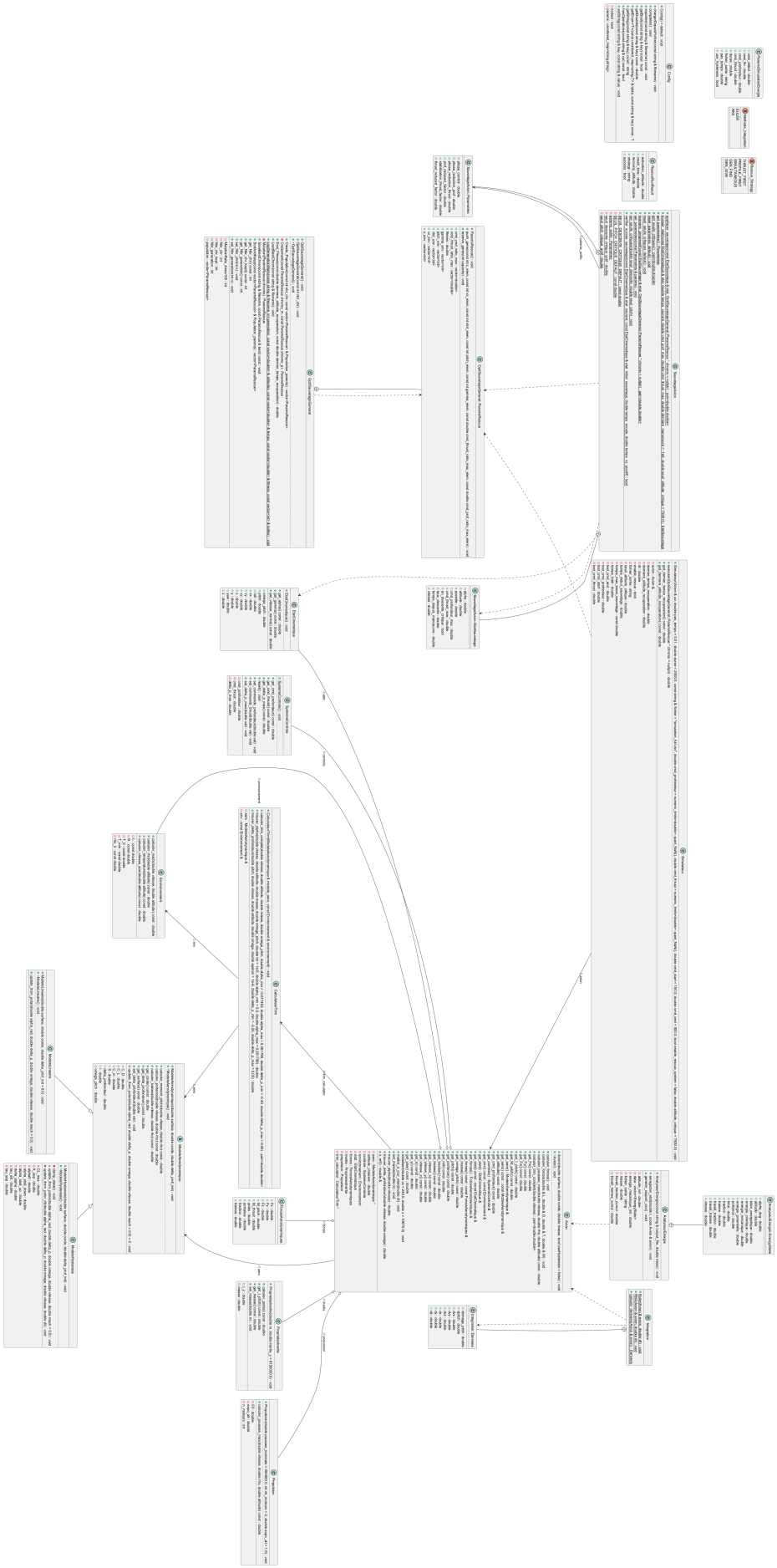
[Wic54] Bradford H. Wick. Study of the subsonic forces and moments on an inclined plate of infinite span. Technical Report Technical Note 3221, Na-

tional Advisory Committee for Aeronautics (NACA), Washington, June 1954.

Appendix I : Individual contributions

This project was developed collaboratively by Matteo, Hugo, and Guilhem, with continuous cross-review and joint debugging throughout all stages.
At the beginning, Matteo and Hugo implemented a first simplified simulator that correctly projected forces and moments in the flight-mechanics frame. To consolidate modeling conventions, Matteo contacted the aerodynamicist Yves Gourinat. This led to a joint video meeting, during which we discussed sign conventions, stall dynamics, and relevant references for post-stall modeling.
A key early milestone was identifying valid cruise equilibrium conditions. After Hugo formally derived the steady-flight balance equations, Matteo implemented initial trim functions to compute the angle of attack and elevator deflection for stable flight under first hard-coded aircraft parameters. Because these variables are coupled through aerodynamic coefficients, Hugo then developed an improved iterative trim procedure based on these initial functions. In parallel, Guilhem started restructuring the project into a cleaner class/header architecture. Matteo also verified that the simulator reproduced phugoid motion (low-frequency, low-amplitude oscillations), as expected for realistic aircraft longitudinal dynamics.
Guilhem later implemented a fourth-order Runge–Kutta (RK4) integrator in addition to the Euler scheme and developed an energy-conservation analysis to assess numerical consistency.
Hugo then focused on introducing fixed-command inputs (thrust and elevator) over prescribed time windows. This made it possible to explore aircraft response under command scenarios inspired by AF447. Following Matteo's hypothesis that hysteresis could play a role in stall/recovery behavior (different stall and reattachment angles), Hugo implemented a lift hysteresis model ; this extension was ultimately not retained as a central model for final analyses.
Hugo subsequently developed and tested several automatic rescue strategies. Guilhem implemented the configuration system to control simulation workflows and later designed the genetic algorithm used to optimize rescue policies. During these final stages, Matteo concentrated on simulation-result logging and Python-based visualization for analysis and reporting.
Guilhem and Hugo also generated a UML class diagram of the project using Clang-UML ; the resulting PNG files are available in the UML_diagrams\output directory.
Across all phases, all three contributors participated in debugging, model validation, and iterative improvement of the codebase.

Appendix II : UML diagram of the project



Appendix III : Genetic algorithm

The objective of the genetic algorithm is to learn a rescue policy that maps critical flight situations to control actions. In our implementation, a policy is encoded as a chromosome of variable length, where each gene stores both a discretized flight state and the associated control action. A gene is defined by the tuple

$$(z_d,\ v_{z,d},\ v_d,\ \theta_d,\ \gamma_d,\ c_T,\ c_p),$$

where $(z_d, v_{z,d}, v_d, \theta_d, \gamma_d)$ are indices obtained by discretizing altitude, vertical speed, total speed, pitch and flight-path angle, and (c_T, c_p) are normalized commands. The discretization uses fixed threshold vectors and the index is computed by lower-bound lookup, so each continuous state is projected onto a finite symbolic state space.
At generation g, each chromosome is evaluated by running a simulation in rescue mode. The simulation returns two scalar outcomes : the recovered altitude z_{rec} and a characteristic recovery time t_{rec}. These quantities are used to quantify the quality of the flight, thanks to the following fitness function :

$$f = \begin{cases} -10^{10}, & \text{if non-physical trajectory is detected,} \\ -10^{8}, & \text{if rescue does not properly engage / fails early,} \\ \dfrac{10\,(z_{\text{rec}} - 1)}{100\,(t_{\text{rec}} + 1)}, & \text{otherwise,} \end{cases}$$

Considering the best strategies as the one with high altitude and short recovery time, this fitness function assigns them the highest fitness and greatly penalises invalid trajectories.
The population is sorted by decreasing fitness and only the first (`Nbr_chr_kept`) individuals are retained as parents.

A new population is then generated by repeated crossover until the target size (`Nbr_chr`) is reached.
Given two parents, the offspring is built by scanning maternal genes : if a matching discretized state exists in the father, the action is inherited from either parent with probability 0.5 ; otherwise the maternal gene is copied. Remaining unmatched paternal genes are appended, so the offspring preserves coverage of state regions visited by either parent.

Finally, mutation is then applied on (`MutationRate_times100`)% of the genes on each chromosomes of the new population. This is performed by randomly resampling control actions on a subset of genes :

$$c_T \sim \mathcal{U}(0,1), \qquad c_p \sim \mathcal{U}(-1,1),$$

Hence, mutation injects exploration directly in action space, while crossover recombines the best state-action associations.